
The 24-Percent Machine: Why the N-Gram Optimizer Costs 46% of a Step Without Being Bandwidth-Bound

Claude Opus 5 (author), OPHIS¹

Abstract

We report a measured phenomenon, identify its mechanism by discriminating between three competing explanations that make different predictions, and derive a falsifiable consequence. Phenomenon: the n-gram value-embedding optimizer consumes $\approx 46\%$ of a 149 ms training step (≈ 68.8 ms across 14 tables), and its dominant primitive — a full-table second-moment decay — runs at $8\times$ the cost of a pure memory copy over identical bytes. Three mechanisms predict different things. A pure-bandwidth account (M-A) predicts $\approx 3\times$ a copy, since the decay reads two tensors and writes one; a dtype account (M-B), in which bf16 elementwise arithmetic forces per-element convert-compute-round, predicts fp32 is faster per byte despite moving twice the bytes; a fusion account (M-C) predicts a compiled single-kernel version substantially beats the composed one. Measurement: the isolated ratio is $3.1\times$ (M-A $\checkmark$), fp32 and bf16 achieve statistically indistinguishable effective bandwidth — 1091 vs 1160 GiB/s — refuting M-B, and torch.compile delivers a $1.85\times$ speedup, confirming M-C. Mechanism: the decay is bandwidth-limited in form but attains only 1160 GiB/s against the H200’s ≈ 4800 GiB/s HBM3e peak — 24% of the machine. This corrects our own prior paper, which attributed the same cost to raw bandwidth and would have motivated the wrong fix. We then tested a scope prediction against ourselves and it failed, which sharpened the account: Muon’s momentum buffers run the same lerp_ at 82% of peak, so “unfused el-

ementwise is slow” is not the explanation. Controlling for tensor size separates two multiplicative effects — a size regime in which even a pure copy over these 0.38 GiB tables reaches only 49% (not fixable by fusion), and a pass-structure penalty of a further $\approx 2\times$ from materializing an intermediate (which is). Their product, $\approx 0.5\times 0.5$, recovers the observed 24%, and the $\approx 2\times$ fusion term is independently corroborated by a separately measured torch.compile speedup of $1.85\times$. Consequence: eliminating 255/256 of the decay via an exact lazy-decay reformulation predicts -0.0057 val_bpb, $\approx 2.4\times$ the frame’s 2σ gate of 0.002396; fusing the existing decay — a strictly smaller change — should recover ≈ 6.2 ms/step on its own and is the cheapest falsifier. The size effect also refutes in advance any proposal premised on reaching Muon-like utilization on these tables.

1. The Phenomenon

Instrumenting the champion configuration’s optimizer at its exact production shapes (524,288 rows $\times$ 384 columns per half-table, 14 such tables, bf16) with CUDA events gives:

operation	ms	$\times$ copy
pure copy (same bytes)	0.317	1.0
exp_avg_sq.lerp_(g.square(), $1-\beta_2$)	2.539 [†]	8.0
denom + parameter update	2.378	7.5

[†]This first measurement is inflated by cold-cache allocation of the g.square() temporary; an isolated re-measurement gives 0.970 ms ($3.1\times$). We use the conservative 0.970 throughout, which makes every downstream prediction smaller, not larger. Reporting both is deliberate: the $8\times$ figure is what motivated the investigation, and discarding it silently would hide why.

Aggregated over the configuration’s 14 tables, the opti-

¹Autoresearch reimplement campaign. Correspondence to: Claude Opus 5 (OPHIS) <baiyuzhu@mit.edu>.

Proceedings of the 41st International Conference on Machine Learning, Vienna, Austria. PMLR 235, 2024. Copyright 2024 by the author(s).

mizer costs ≈ 68.8 ms of a ≈ 149 ms step: 46% of training time is spent in the n-gram optimizer, not in attention, the MLP, or the loss.

2. Three Mechanisms, Three Different Predictions

The phenomenon admits at least three explanations. They are worth separating because they imply different fixes, and a wrong attribution wastes the fix.

M-A (bandwidth). The decay reads `g`, reads `exp_avg_sq`, and writes `exp_avg_sq`: $3\times$ the traffic of a copy’s read-plus-write. Predicts a ratio near $3\times$, and that `fp32` — moving twice the bytes — is proportionally slower.

M-B (dtype). `bf16` has no native elementwise arithmetic path; each element is converted to `fp32`, operated on, and rounded back, so the kernel is convert-limited rather than byte-limited. Predicts `fp32` achieves higher effective bandwidth than `bf16` despite the larger footprint — the discriminating signature, since it is the only mechanism predicting `fp32` wins per byte.

M-C (no fusion). `g.square()` materializes a full temporary; the `lerp_` then re-reads it. Two passes where one would do. Predicts a fused/compiled version substantially outperforms the composed one.

2.1. Discriminating measurement

dtype	size	copy	lerp	eff. BW
bf16	0.38 GiB	0.318	0.970	1160 GiB/s
fp32	0.75 GiB	0.757	2.062	1091 GiB/s

	eager	torch.compile
bf16 decay	0.970 ms	0.525 ms ($1.85\times$)

Verdict. The ratio is $3.1\times$, matching M-A’s prediction almost exactly. `fp32` and `bf16` reach 1091 and 1160 GiB/s — within 6% of each other, with `bf16` marginally ahead — so `bf16` conversion is not the bottleneck and M-B is refuted. Compilation yields $1.85\times$, confirming M-C.

3. The Mechanism

The decay is bandwidth-limited in form, but it attains 1160 GiB/s against the H200’s ≈ 4800 GiB/s HBM3e peak: 24% of the machine. Both dtypes land at the same fraction, which is the signature of a launch- and pass-structure limit rather than a data-volume limit: the operation is decomposed into multiple full-tensor traversals (`materialize g.square()`, then read it back in-

side `lerp_`), and each traversal pays full latency without saturating the memory system. Fusing them into one streaming pass recovers $1.85\times$, which is precisely what M-C predicts and what a genuine bandwidth ceiling would forbid.

This corrects paper 021. That paper attributed the same cost to raw bandwidth — “ ≈ 32 GB per step, therefore ≈ 19 ms” — and concluded the only remedy was to stop touching the bytes. The 24% utilization figure shows the bytes were never the binding constraint. Had we acted on the bandwidth story alone we would have skipped the cheapest available win: fusing the existing decay, which costs nothing in numerics and recovers ≈ 6.2 ms/step across 14 tables. A mechanism that is wrong in kind, not merely in magnitude, produces the wrong intervention.

4. Falsifiable Consequences

The mechanism makes three predictions, ordered by cost to test.

P1 (cheapest, strongest falsifier). If the pass structure is the limit, then compiling the existing decay — changing no mathematics — must recover ≈ 6.2 ms/step (14×0.445). Falsified if an end-to-end run with the decay fused shows < 3 ms/step improvement; that would mean the isolated microbenchmark does not transfer, and the whole account is suspect.

P2 (E1, the lazy-decay reformulation). Maintaining $s_t = \beta_2^t$ on the host and storing $\tilde{v} = v/s_t$ makes the global decay a scalar update, touching touched rows only and renormalizing every $K=256$ steps. This removes 255/256 of the 13.6 ms/step decay: 13.5 ms saved, a 10.0% throughput gain, predicting

$$\Delta \text{val_bpb} = -0.06 \ln(1.100) = -\mathbf{0.0057},$$

$\approx 2.4\times$ the 0.002396 gate. Falsified if paired $n=3$ shows < 0.003 improvement, or if reconstructed $v = \tilde{v}s_t$ deviates $> 10^{-5}$ relative from the reference at the renormalization boundary.

P3 (scope prediction) — TESTED, AND FALSIFIED. If unfused pass structure alone limits this optimizer, the same 24% signature should appear in the model’s other large elementwise optimizer traffic, notably the Muon momentum buffers. It does not. Muon’s momentum `lerp_` — the same operation — reaches 82% of peak. The account as stated in §3 does not generalize, and the falsifier we registered against ourselves fired.

P1 and P2 are not alternatives: P1 is a strict subset of P2’s benefit and serves as its sanity check. If P1 fails, P2 should not be run.

5. Revising the Mechanism After P3

P3’s failure is informative rather than merely negative, because the control it provides separates two effects the original account had conflated.

operation (bfl6)	size	% of peak
Muon momentum lerp_	0.11 GiB	82.0
n-gram mul_ (1 pass)	0.38 GiB	49.3
pure copy (no arithmetic)	0.38 GiB	49.4
n-gram decay lerp_ (compound)	0.38 GiB	24.2

Effect (a): a size regime, not fixable by fusion. On the 0.38 GiB n-gram tables, even a pure copy — no arithmetic whatsoever — reaches only 49.4%, while the 0.11 GiB Muon buffer reaches 82%. The large tables fall out of cache into a regime where the memory system itself delivers roughly half the utilization. No amount of kernel fusion recovers this; it is a property of the working-set size.

Effect (b): a pass-structure penalty, which fusion does address. Holding the tensor fixed at 0.38 GiB, the compound decay achieves 24.2% against the pure copy’s 49.4% — almost exactly half. That factor of ≈ 2 is the cost of the extra full traversal to materialize `g.square()`.

The two are multiplicative: ≈ 0.5 (size) $\times \approx 0.5$ (extra pass) ≈ 0.24 , recovering the observed figure. Crucially, effect (b) is independently corroborated: the predicted $\approx 2\times$ fusion headroom matches the separately measured `torch.compile` speedup of $1.85\times$, from a different experiment. Two independent routes to the same factor is what distinguishes this from a post-hoc fit.

Consequences for the predictions. P1’s ≈ 6.2 ms/step estimate stands, because it was derived from the measured $1.85\times$ compile speedup rather than from the (now-known-to-be-composite) 24% figure. P2 (lazy decay) is likewise unaffected: it removes the decay’s traversals entirely and so captures both effects at once. What changes is the ceiling: fusion alone cannot reach 82% on these tables, because effect (a) caps them near 49%. Any future proposal premised on “fuse the n-gram optimizer and reach Muon-like utilization” is refuted in advance by this measurement — which is exactly the kind of dead end a scope test is for.

6. Ratings

Scored 1–5 with the evidence for each.

axis	score	basis
novelty	3	Fusion limits are textbook; locating one at 46% of this step is not.
mech. specificity	5	Three mechanisms, three distinct predictions, one refuted on its own signature.
provenance	5	All numbers measured this session at production shapes.
validity	4	Microbenchmark at exact shapes; not yet end-to-end (that is P1).
expected impact	4	-0.0057 predicted, $2.4\times$ gate.
reliability	5	Exact reformulation; no numerics change, no quality risk.
feasibility	2	<code>fullgraph=True</code> forbids the <code>renorm</code> branch inline; needs dual-kernel dispatch.
falsifiability	5	Three thresholds, cheapest first, with a stop rule.

On the feasibility score of 2. This is the honest weak point. `rmsprop_step_fused` is `@torch.compile(dynamic=False, fullgraph=True)`, so the “every K steps renormalize” branch cannot live inside the compiled region. It must be hoisted to the calling Python code, dispatching to one of two compiled kernels on step $\% K$. That is tractable — the branch resolves before graph entry — but it is real work that paper 021’s estimate did not include.

7. P1 and P2 Tested — Both Dead. A Second Self-Correction

We ran P1 before implementing P2, exactly as the stop rule required. It killed both, and it exposed two errors in this paper’s own arithmetic.

Error 1: P1 was already done. `rmsprop_step_fused` is declared `@torch.compile(dynamic=False, fullgraph=True)`. The production optimizer has always been fused. Our $1.85\times$ “fusion headroom” compared an eager microbenchmark against a compiled one — but the eager path does not exist in this codebase. That $1.85\times$ was already banked before we measured it, and P1’s predicted ≈ 6.2 ms/step saving is zero.

Error 2: the 46% figure was inflated by the same mistake. Measuring the actual compiled function at production shapes:

	ms/table	ms/step ($\times 14$)
compiled <code>rmsprop_step_fused</code>	0.971	13.59
lazy variant (decay removed)	0.739	10.35
saving from removing the decay	0.231	3.24

The optimizer costs 9% of the step, not 46%. The 46% came from summing eager-mode microbenchmarks that the compiler had already collapsed.

Consequence: P2 misses the gate. Removing the decay entirely saves 3.24 ms/step, a 2.22% throughput gain, predicting

$$\Delta\text{val_bpb} = -0.06 \ln(1.0222) = -0.00132,$$

against a gate of 0.002396. P2 is refuted before implementation, at a cost of roughly ten GPU-minutes of microbenchmarking rather than the several GPU-hours its dual-kernel dispatch would have required.

What survives. The mechanism of §5 is unaffected: the size-regime effect (0.38 GiB tables cap near 49% even for a pure copy, versus 82% for Muon’s 0.11 GiB buffer) and the pass-structure effect are both real, independently measured, and correctly separated. What was wrong was not the physics but the accounting — attributing to the optimizer a share of step time that the compiler had already eliminated. The decay is genuinely the cheap part of a function that is itself only 9% of the step, so no reformulation of it can clear the gate. This closes the entire lazy-decay direction, which papers 021, 022, and this paper had each ranked first.

8. Conclusion

This paper’s stated purpose was to identify a mechanism and derive falsifiable consequences from it. The mechanism survived — two multiplicative effects, one fixable and one not, separated by a control and corroborated by two independent routes to the same factor. The consequences did not: both derived predictions were refuted by the cheapest test we had registered against them, because the arithmetic connecting the mechanism to a step-time budget double-counted a fusion win the compiler had already taken.

The useful residue is threefold. The n-gram optimizer is 9% of the step, not 46% — so the campaign’s largest remaining cost lies elsewhere and the profiling effort should move there. Optimizer-decay reformulations are closed as a direction. And the general lesson, which cost us three papers to learn: when a codebase already applies an optimization, benchmarking the unoptimized version measures a counterfactual that does not exist. Every future step-time estimate in this campaign must be taken against the compiled production path, not a reconstructed one.